

check_cm_test_mock_pid_patch ed_when_real_pid

scripts/pre_build/ check_cm_test_mock_pid_patch ed_when_real_pid.sh — CM-TEST- MOCK-PID-PATCHED-WHEN- REAL-PID static pre-build gate

Revision: 1 **Last modified:** 2026-08-19T17:00:43Z **Status:** active **Item:** BOB-128 (Task 8 syscall-audit recommendation #1, §11.4.238 discovery-channel-escape closure — a new automated check for the adjacent-class defect BOB-127 fixed at commit 8bedc5a)

Purpose

scripts/pre_build/check_cm_test_mock_pid_patched_when_real_pid.sh statically detects TEST files that construct a subprocess-shaped AsyncMock() / MagicMock(), set an explicit **real integer-literal** .pid = <int> on it (the sanctioned CM-TEST-MOCK-PID-EXPLICIT-INT hardening pattern — see that gate's own doc), but never neutralise the actual destructive syscall by patching os.killpg **specifically**.

The defect class this closes (BOB-127)

CM-TEST-MOCK-PID-EXPLICIT-INT closes the CATASTROPHIC class: a mock whose `.pid` is left unconfigured coerces to the integer 1, and `os.killpg(os.getpgid(1), SIGKILL)` is, under glibc, identical to `kill(-1, SIGKILL)` — a broadcast kill of every process the caller's UID owns (BOB-116/120/123/124/125/126).

Satisfying that gate — giving the mock an explicit real `int > 1 pid` — is **necessary but not sufficient**. The production guard in `download-proxy/src/merge_service/search.py` is:

```
_pid = proc.pid
if isinstance(_pid, int) and _pid > 1:
    with contextlib.suppress(Exception):
        _pgid = os.getpgid(_pid)
        if isinstance(_pgid, int) and _pgid > 1:
            os.killpg(_pgid, _signal.SIGKILL)
```

Any real positive `int` satisfies `isinstance(_pid, int) and _pid > 1` — so a test that sets `mock.pid = 12345` (or any other real `int`) but does **not** patch `os.killpg/os.getpgid` lets the code proceed past the guard and call the **REAL** `os.getpgid(<pid>)` and, if that resolves, the **REAL** `os.killpg(<pgid>, SIGKILL)` against whatever process/process-group happens to own that literal, hardcoded, non-test-owned PID on the host actually running the test.

This is not the CATASTROPHIC `kill(-1, ...)` class (a real `pid` resolving to `pgid 1` is astronomically unlikely) — it is DANGEROUS: Linux PID reuse (`pid_max 32768` by default, lower on many container/desktop configs) means a hardcoded literal PID **can** legitimately belong to a live, unrelated process at test-run time, especially on a long-running dev host with many processes — exactly the operator's environment where the BOB-126 incidents happened. If that collision occurs, an unpatched test fires a real `SIGKILL` at a real, unrelated process group: collateral damage from a “unit” test.

Forensic anchor: Task 8's syscall audit (`.superpowers/sdd/task-8-syscall-audit.md`, DANGEROUS findings #1 and #2) found exactly this shape live in `tests/unit/merge_service/test_public_tracker_subprocess_timeout.py`: `test_stuck_subprocess_killed_and_abandoned` (`proc.pid = 12345`) and `test_cleanup_timeout_abandons_zombie` (`proc.pid = 1111`), neither patching `os.killpg/os.getpgid`. Both were fixed at commit `8bedc5a` by copying the already-correct pattern from the third test in the same file, `test_process_group_kill_called_on_deadline` (`proc.pid = 9999`), which

patches both `_search.os.getpgid` and `_search.os.killpg` before exercising the same code path. This gate makes that “explicit pid ⇒ ALSO patch `os.killpg`” pairing a **mechanically enforced invariant** instead of tribal knowledge, per §11.4.226 (evidence-class-at-closure) and §11.4.240/§11.4.249 (an independent, structurally separate check — never the fix’s own author-supplied oracle).

Usage

```
# Default scope: tests/**/*.py (relative to the repo root this
    script
# resolves from its own location), excluding tests/pre_build/
# (which authors deliberately dangerous-shaped fixtures for its
    own
# meta-tests – flagging them would be a false positive on the
# enforcement harness itself).
scripts/pre_build/check_cm_test_mock_pid_patched_when_real_pid.sh

# Explicit-path mode – scan exactly the given file(s)/
    directory(ies)
# instead of the default scope. Used by the meta-test to run
    against
# hermetic fixtures.
scripts/pre_build/
    check_cm_test_mock_pid_patched_when_real_pid.sh /path/to/
    file.py
scripts/pre_build/
    check_cm_test_mock_pid_patched_when_real_pid.sh /path/to/
    fixture/dir

# Verbose (prints every hit that was skipped/exempted, not just
    findings):
scripts/pre_build/check_cm_test_mock_pid_patched_when_real_pid.sh
    -v

scripts/pre_build/check_cm_test_mock_pid_patched_when_real_pid.sh
    --help
```

Inputs

- **Positional arguments** (optional): zero or more file/directory paths.
 - Zero arguments → default scope: `tests/` under the repository root, walked recursively, excluding `tests/pre_build/`.
 - One or more arguments → scan exactly those paths instead (single files must end in `.py` to be considered; directories are walked with the same directory-name excludes as the default scope).

- **-v / --verbose**: also print each hit that was skipped (no `.returncode = None`, no lifecycle indicator, no pid-literal) or exempted (a qualifying `os.killpg` patch found nearby).
- **-h / --help**: print the in-source documentation header and exit 0.
- No environment variables are read.

Outputs

- **stdout**: a run header (scope, window sizes, file count) and either `PASS: CM-TEST-MOCK-PID-PATCHED-WHEN-REAL-PID clean across N file(s) or (on -v) skip (...) / exempt (...) lines per non-flagged hit`.
- **stderr** (only on failure): a `=== FINDINGS ===` block, one `FAIL:`
`<path>:<line>: real-pid subprocess-mock with NO os.killpg patch`
`- <trimmed source> [identifier=...] line per unpatched hit,`
followed by a `FAIL: <N> unpatched real-pid test-subprocess-mock`
`hit(s) ... summary line`.
- **Exit codes**:
 - 0 — zero unpatched real-pid hits (or, in explicit-path mode, zero matching files — an empty scope is not itself a finding).
 - 1 — one or more unpatched hits; see stderr for the per-hit report.
 - 2 — usage/environment error (unknown flag, or an explicit `PATH` argument that does not exist). Distinct from finding-level `FAIL`, per §11.4.201(4) “conservative-safe default on an unresolvable signal”.

Detection design

Shares its reachability precondition with `CM-TEST-MOCK-PID-EXPLICIT-INT` (see that gate’s own header for the full evidence trail proving `.returncode = None` is the correct, non-guessed precondition for `search.py`’s kill branch):

1. A bare-variable assignment `<ident> = AsyncMock()` or `<ident> = MagicMock()` on its own line.
2. Within the 20 lines immediately AFTER that assignment (the “forward window”), the SAME identifier carries **all** of:
 1. an explicit `<ident>.returncode = None` assignment (required precondition — a mock that can never reach “still running” can never reach the kill branch at all);

2. at least one process-lifecycle/streaming indicator:
`<ident>.stdout.readline, <ident>.stderr.read, <ident>.wait,`
`<ident>.kill;`
3. **an explicit `<ident>.pid = <int-literal> assignment`** — this is what distinguishes this gate's SCOPE from CM-TEST-MOCK-PID-EXPLICIT-INT: a mock with NO pid-literal at all is that sibling gate's job, not this one, and is silently skipped here (never double-flagged by both gates).
3. The identifier does NOT carry a real (non-comment) patch of `os.killpg` **specifically** within 30 lines before or after the mock-assignment line.

Why this gate's exemption is *narrower* than CM-TEST-MOCK-PID-EXPLICIT-INT's

CM-TEST-MOCK-PID-EXPLICIT-INT's exemption accepts a real patch of **any** module's attribute literally named "killpg" — safe for that gate's threat model (a mock coercing to 1; a patch of *something* called killpg nearby is corroborating evidence a human deliberately neutralised the call). That looser match is **not** safe here: this gate's whole point is verifying the syscall the code *actually* reaches (`os.killpg`) is neutralised. A patch of an unrelated `some_other_module.killpg` attribute does nothing to stop the real `os.killpg` call and must NOT exempt a hit — accepting it would be a §11.4.201 false-positive-refusal-shaped bug at the gate layer (the near -carrier footgun: a token that *mentions* the pattern matched as if it *were* the pattern, per §11.4.196(D)/§12.12/§11.4.201(6)-(7)).

os.killpg-targeting exemption

A hit is SKIPPED (not flagged) when a REAL (non-comment) line within ±30 lines of the mock-assignment line matches one of:

- **String-literal form:** `patch("<optional-dotted-prefix.>os.killpg") / patch('<optional-dotted-prefix.>os.killpg')` — the target string's last two dotted segments must be exactly `os.killpg` (`patch("os.killpg")` and `patch("merge_service.os.killpg")` both count; `patch("some_other_module.killpg")` does **not** — there is no `os.` immediately before `killpg`).
- **Object form:** `patch.object(<optional-dotted-prefix.>os, "killpg") / patch.object(<optional-dotted-prefix.>os, 'killpg')` — the first argument is either the bare name `os` or a dotted attribute chain ENDING in `.os` (the real-tree pattern this gate was

written against: `patch.object(_search.os, "killpg")`), and the second argument is the string literal `killpg`.

Both forms anchor the literal token `os` at a `.`-or-start-of-token boundary — a target that merely *contains* the substring `"os"` (e.g. `chaos.killpg`) does not match. A patch mentioned only inside a comment is never counted.

Side-effects

None. The gate is read-only: it never modifies any scanned file. It creates one `mktemp` findings file for its own run and removes it on exit via `trap ... EXIT`.

Dependencies

- `bash` (uses `[[ ]]`, `<<<` here-strings, process substitution).
- `grep -E`, `sed -E`, `find`, `wc` — all standard on the project's supported hosts.
- No Python, no network access, no repository write access.

Cross-references

- **\$11.4.263** — process-group signal-safety mandate: the anchor this whole gate family (`CM-KILLPG-PGID-GUARD`, `CM-TEST-MOCK-PID-EXPLICIT-INT`, `CM-TEST-MOCK-PID-PATCHED-WHEN-REAL-PID`) enforces.
- **BOB-126 / BOB-127** — BOB-126 is the original 7-incident broadcast-kill forced-logout chain; BOB-127 is the adjacent DANGEROUS finding Task 8's syscall audit surfaced and commit `8bedc5a` fixed — the exact defect class this gate closes.
- `scripts/pre_build/check_cm_killpg_pgid_guard.sh / docs/scripts/check_cm_killpg_pgid_guard.md` — the PRODUCTION-side sibling gate (guards `download-proxy/src/`, `plugins/`, `scripts/`).
- `scripts/pre_build/check_cm_test_mock_pid_explicit_int.sh / docs/scripts/check_cm_test_mock_pid_explicit_int.md` — the closest sibling: guards against a mock with NO pid-literal at all (coerces-to-1). This gate is its adjacent-class extension: guards the case where a pid-literal IS present but the syscall is still unmocked.
- `.superpowers/sdd/task-8-syscall-audit.md` — the audit report whose recommendation #1 this gate implements verbatim (including the suggested golden-good/golden-bad fixture pairing).

- **§11.4.238** — QA-discovery-channel mandate: this gate is the “new or strengthened automated check” a coverage-escape audit requires for a defect first found out-of-band (Task 8’s manual audit, not a standing automated check).
- **§11.4.226** — evidence-class-at-closure: a standing guard, not merely a source-side patch in two test functions, is what keeps this defect class from silently reopening the next time someone authors a subprocess-mock test.
- **§11.4.115 / §11.4.135** — RED-first meta-test discipline / permanent regression-guard suite: tests/pre_build/test_check_cm_test_mock_pid_patched_when_real_pid.sh is this gate’s paired §1.1 mutation harness.
- **§11.4.196(D) / §12.12 / §11.4.201(6)-(7)** — the “instrument matches a MENTION of its own carrier text, not the real pattern” footgun class the narrower os.killpg-targeting exemption (vs. the sibling gate’s any-module exemption) closes.
- **§11.4.201** — every guard/gate asserts the real condition; a false-positive refusal (accepting an irrelevant patch as an exemption) is a FAIL-bluff exactly as forbidden as a false-negative pass.
- tests/unit/merge_service/test_public_tracker_subprocess_timeout.py — contains both the ORIGINAL pre-fix defect shape (before commit 8bedc5a) and the sanctioned good-pattern reference (test_process_group_kill_called_on_deadline), all in one file.

Companion files

- scripts/pre_build/check_cm_test_mock_pid_patched_when_real_pid.sh — the gate itself.
- tests/pre_build/test_check_cm_test_mock_pid_patched_when_real_pid.sh — the §1.1 paired meta-test: five hermetic fixtures — golden-good-1 (.pid = 12345 + object-form patch.object(<mod>.os, "killpg")), golden-good-2 (.pid = 12345 + string-literal-form patch("os.killpg")), golden-bad-1 (.pid = 12345, no killpg patch — the pre-8bedc5a shape), golden-bad-2 (.pid = 1111, no killpg patch — proves detection isn’t hardcoded to 12345), false-positive-guard (.pid = 12345 + patch("some_other_module.killpg") — must still FAIL, per §11.4.201) — plus a real-tree smoke run and a no-op-stub negative control.

Example fixtures

Golden-good (properly hardened — gate PASSes):

```
proc = AsyncMock()
proc.returncode = None
proc.pid = 9999
proc.stdout = MagicMock()
proc.stdout.readline = AsyncMock(side_effect=_hang)
proc.kill = MagicMock()

with (
    patch("asyncio.create_subprocess_exec", return_value=proc),
    patch.object(_search.os, "getpgid", return_value=9999),
    patch.object(_search.os, "killpg") as mock_killpg,
):
    ...
```

Golden-bad (the BOB-127 shape — gate FAILs):

```
proc = AsyncMock()
proc.returncode = None
proc.pid = 12345          # a real int satisfies the production
                        guard
proc.stdout = MagicMock()
proc.stdout.readline = AsyncMock(side_effect=_hang)
proc.kill = MagicMock()

with (
    patch("asyncio.create_subprocess_exec", return_value=proc),
    # NO patch of os.killpg / os.getpgid – the REAL syscall fires
    # against the hardcoded PID 12345 if it happens to be live.
):
    ...
```

Last verified

2026-08-19 — bash tests/pre_build/
test_check_cm_test_mock_pid_patched_when_real_pid.sh ran GREEN
(7/7: golden-good-1 rc=0, golden-good-2 rc=0, golden-bad-1 rc=1, golden-bad-2 rc=1, false-positive-guard rc=1, real-tree default scope rc=0, no-op-stub negative control). Real-tree run against the current checkout confirms all 5 real-pid reachable-kill-branch subprocess mocks in tests/**/*.py (test_public_tracker_subprocess_timeout.py x3, test_deadline_tunable.py x1, test_search_deep_coverage.py x1) are correctly recognised as exempt (properly os.killpg-patched) —

verified with -v output. §11.4.115 RED-first evidence captured separately (a golden-bad-shaped fixture placed at /tmp, outside the tracked tree, correctly FAILED the gate before being deleted).